



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Estudios Superiores Acatlán
Licenciatura en Ciencia de Datos

Proyecto Final: Zettelkasten Multi-DBMS

Bases de Datos no Estructuradas
Profesor: M. en C. Pablo Martínez Castro

Alumnos:
Luis Alfredo Ramírez Maza
Bruno Ruiz Juárez

19 de mayo de 2026

Índice

1	Introducción	2
2	Arquitectura del Sistema	2
2.1	Modelo de Persistencia Políglota	2
2.2	Orquestación con Docker Compose	2
2.3	Flujo ETL	3
3	Capa Relacional: PostgreSQL	3
3.1	Justificación	3
3.2	Esquema	3
3.3	Ejemplo de Población	4
4	Capa Documental: MongoDB	4
4.1	Justificación	4
4.2	Estructura del Documento	5
5	Capa de Grafos: Neo4j	5
5.1	Justificación	5
5.2	Modelo de Propiedades	5
5.3	Consultas Características	6
6	Capa Analítica: ClickHouse	6
6.1	Justificación	6
6.2	Tablas Materializadas	6
6.3	Conexión a Metabase	6
7	Resultados Analíticos: Dashboard de Metabase	7
7.1	Distribución por Estado	7
7.2	Distribución por Materia	7
7.3	Top 10 de Temas	8
7.4	Comparativa Interanual	8
7.5	Distribución de Tipos de Enlace	9
8	Benchmarks: Comparativa Multi-DBMS	10
8.1	Metodología	10
8.2	Q1: Agregación OLAP	10
8.3	Q2: Documento Rico	10
8.4	Q3: Recorrido de Grafo	11
8.5	Q4: Ranking Analítico	11
8.6	Síntesis del Benchmark	12
9	Conclusiones	12

1. Introducción

El paradigma *Zettelkasten*, popularizado por el sociólogo Niklas Luhmann, propone un sistema de gestión del conocimiento basado en notas atómicas interconectadas mediante enlaces semánticos tipificados. Su materialización digital plantea, sin embargo, un reto técnico de fondo: la información que produce un *Zettelkasten* es intrínsecamente heterogénea. Coexisten datos estrictamente transaccionales (autoría, integridad referencial, control de versiones), contenido semánticamente flexible (cuerpo *markdown*, metadatos arbitrarios por nota), una topología de grafo densa (*links*, *backlinks*, *prerrequisitos*) y, finalmente, una capa de explotación analítica que demanda agregaciones masivas sobre miles de eventos históricos.

2. Arquitectura del Sistema

2.1. Modelo de Persistencia Políglota

La arquitectura se organiza en torno a un *system of record* relacional (PostgreSQL) que actúa como fuente única de verdad. A partir de él, tres procesos ETL en Python materializan vistas optimizadas hacia los tres motores secundarios. Esta estrategia garantiza tres propiedades críticas: (i) consistencia eventual sobre todas las réplicas; (ii) reproducibilidad determinista del estado del sistema; (iii) capacidad de regenerar cualquier capa derivada sin comprometer la integridad del *system of record*.

2.2. Orquestación con Docker Compose

La totalidad de los servicios se levanta mediante un único archivo `docker-compose.yml`, lo cual aporta tres ventajas operáticas: aislamiento de dependencias, portabilidad entre máquinas y reproducibilidad exacta de las versiones empleadas (PostgreSQL 16, MongoDB 7, Neo4j 5 Community y ClickHouse 24.8). El listado 1 muestra un fragmento representativo.

```
1 services:
2   postgres:
3     image: postgres:16
4     container_name: zettel_postgres
5     environment:
6       POSTGRES_DB: zettelkasten
7       POSTGRES_USER: zettel_user
8       POSTGRES_PASSWORD: zettel_pass
9     ports: ["5432:5432"]
10    volumes:
11      - postgres_data:/var/lib/postgresql/data
12
13   mongo:
14     image: mongo:7
15     ports: ["27017:27017"]
16
17   neo4j:
18     image: neo4j:5-community
19     ports: ["7474:7474", "7687:7687"]
20
21   clickhouse:
```

```

22     image: clickhouse/clickhouse-server:24.8
23     ports: ["8123:8123", "9000:9000"]
24
25     metabase:
26         image: metabase/metabase:latest
27         ports: ["3000:3000"]
28         depends_on: [clickhouse]

```

Listing 1: Fragmento del archivo `docker-compose.yml`.

2.3. Flujo ETL

El conjunto de scripts numerados implementa una cadena lineal de transformación:

1. `01_seed_postgres.py`: población determinista de PostgreSQL con 1 200 notas, 25 usuarios, 80 *tags*, 250 fuentes, 3 000 *links* y 5 000 eventos, empleando `random.seed(42)` para garantizar la reproducibilidad.
2. `02_postgres_to_mongo.py`: desnormalización hacia documentos JSON con datos embebidos (*autor*, *materia*, *tags*, *links*, *backlinks*, *versiones* y *eventos*).
3. `03_postgres_to_neo4j.py`: proyección del esquema relacional como grafo de propiedades (*property graph*).
4. `04_postgres_to_clickhouse.py`: *flattening* hacia tablas columnar preagregadas optimizadas para OLAP.
5. `06_benchmark_multi_dbms_v2.py`: ejecución cronometrada de las cuatro consultas canónicas sobre los cuatro motores.

3. Capa Relacional: PostgreSQL

3.1. Justificación

PostgreSQL actúa como núcleo transaccional del sistema por tres motivos sustantivos: garantías ACID nativas, soporte robusto de claves foráneas para preservar la integridad referencial y el tipo de dato JSONB, que admite metadatos semiestructurados sin sacrificar la rigidez relacional general.

3.2. Esquema

El modelo se compone de doce relaciones que cubren tanto las entidades nucleares (*usuario*, *materia*, *tema*, *nota*, *tag*, *fuentes*) como las relaciones (*nota_tag*, *nota_fuentes*, *nota_link*), versionado (*nota_version*), bitácora (*nota_evento*) y consultas registradas (*nota_consulta*). La tabla central, *nota*, conserva los campos canónicos del Zettelkasten: *titulo*, *slug*, *contenido*, *tipo*, *estado*, *fecha_creacion* y *fecha_actualizacion*, además de claves foráneas hacia *usuario* y *tema*.

La tabla de enlaces, *nota_link*, modela la topología del *Zettelkasten* mediante una relación dirigida y tipificada entre dos notas, incorporando atributos semánticos (*tipo_link*, *contexto*, *peso*) que enriquecen la semántica del enlace más allá de una simple adyacencia.

materia, sus *tags*, sus *links* salientes, sus *backlinks*, su historial de versiones y sus eventos. Esto reduce drásticamente la latencia de las consultas de tipo *document fetch*, típicas de la vista frontal de cualquier aplicación Zettelkasten.

4.2. Estructura del Documento

El proceso `02_postgres_to_mongo.py` construye documentos con la siguiente jerarquía anidada:

```
1 documento = {
2     "titulo": ...,
3     "slug": ...,
4     "contenido": ...,
5     "tipo": ..., "estado": ...,
6     "fechas": {"creacion": ..., "actualizacion": ...},
7     "autor": {"nombre": ..., "email": ..., "rol": ...},
8     "materia": {
9         "nombre": ...,
10        "tema": {"nombre": ..., "descripcion": ...}
11    },
12    "tags": [ {...}, ... ],
13    "fuentes": [ {...}, ... ],
14    "links": [ {...}, ... ],      # salientes
15    "backlinks": [ {...}, ... ], # entrantes
16    "versiones": [ {...}, ... ],
17    "eventos": [ {...}, ... ]
18 }
```

Listing 3: Estructura del documento JSON en MongoDB.

5. Capa de Grafos: Neo4j

5.1. Justificación

La topología natural de un Zettelkasten es la de un grafo dirigido con aristas tipificadas. Las preguntas analíticamente interesantes (¿qué notas son prerequisite de un concepto?, ¿qué camino conecta dos ideas separadas por tres saltos?, ¿cuál es la centralidad de una nota?) son intratables sobre un modelo relacional sin recurrir a **Common Table Expressions** recursivas con un costo combinatorio prohibitivo. Neo4j resuelve estas consultas en tiempo proporcional al tamaño del subgrafo recorrido, no al tamaño total de la base.

5.2. Modelo de Propiedades

El proceso `03_postgres_to_neo4j.py` proyecta seis tipos de nodos (Usuario, Materia, Tema, Nota, Tag, Fuente) y siete tipos de relaciones (:CREO, :ES PARTE DE, :PERTENECE A, :TIENE TAG, :USA FUENTE y :LINK con atributos *tipo*, *peso*, *contexto*). Además, se imponen restricciones de unicidad sobre las propiedades identificadoras de cada nodo.

```
1 UNWIND $rows AS row
2 MATCH (origen:Nota {id_nota: row.id_nota_origen})
3 MATCH (destino:Nota {id_nota: row.id_nota_destino})
4 MERGE (origen)-[r:LINK {id_link: row.id_link}]->(destino)
```

```

5 SET r.tipo           = row.tipo_link,
6   r.contexto         = row.contexto,
7   r.peso              = row.peso,
8   r.fecha_creacion    = row.fecha_creacion

```

Listing 4: Carga de enlaces tipificados en Neo4j.

5.3. Consultas Características

Los recorridos de longitud variable, expresables en Cypher mediante el operador `[ :LINK*1..4 ]`, son el caso de uso paradigmático donde Neo4j domina al resto de motores. Estos recorridos resultan inviables sobre PostgreSQL para profundidades crecientes y carecen de soporte nativo en MongoDB y ClickHouse.

6. Capa Analítica: ClickHouse

6.1. Justificación

Las consultas analíticas canónicas sobre un Zettelkasten (distribución de notas por estado/materia, comparativos anuales, *top-N* de temas, distribución de tipos de enlace) son ejemplos clásicos de carga OLAP: lecturas masivas, agregaciones por pocas dimensiones, predicados poco selectivos. ClickHouse, motor columnar basado en *MergeTree*, está diseñado específicamente para este perfil: lee únicamente las columnas referenciadas, comprime de forma agresiva y vectoriza la ejecución.

6.2. Tablas Materializadas

El proceso `04_postgres_to_clickhouse.py` produce un esquema en estrella simplificado con cinco tablas preagregadas:

- `nota_analytics`: una fila por nota, con conteos pre-calculados de *tags*, *links* salientes y *backlinks*.
- `link_analytics`: una fila por enlace, con denormalización de los títulos y *slugs* de origen y destino.
- `tag_analytics`: tupla *nota-tag* para consultas de coocurrencia.
- `evento_analytics`: bitácora de eventos para análisis temporal.
- `benchmark_analytics`: tabla de resultados de los propios *benchmarks*.

La preagregación de columnas costosas (`total_tags`, `total_links_salientes`, `total_backlinks`) durante el ETL elimina el costo de los JOIN en tiempo de consulta, desplazándolo a tiempo de carga.

6.3. Conexión a Metabase

Sobre la capa columnar, Metabase se configura como cliente de visualización. El conector nativo permite construir el *dashboard* sin escribir SQL adicional: los gráficos se alimentan directamente de las tablas `nota_analytics` y `link_analytics`.

7. Resultados Analíticos: Dashboard de Metabase

El *dashboard* construido sobre ClickHouse expone seis visualizaciones derivadas de las 1 200 notas y 3 000 enlaces del sistema. Esta sección sintetiza los hallazgos.

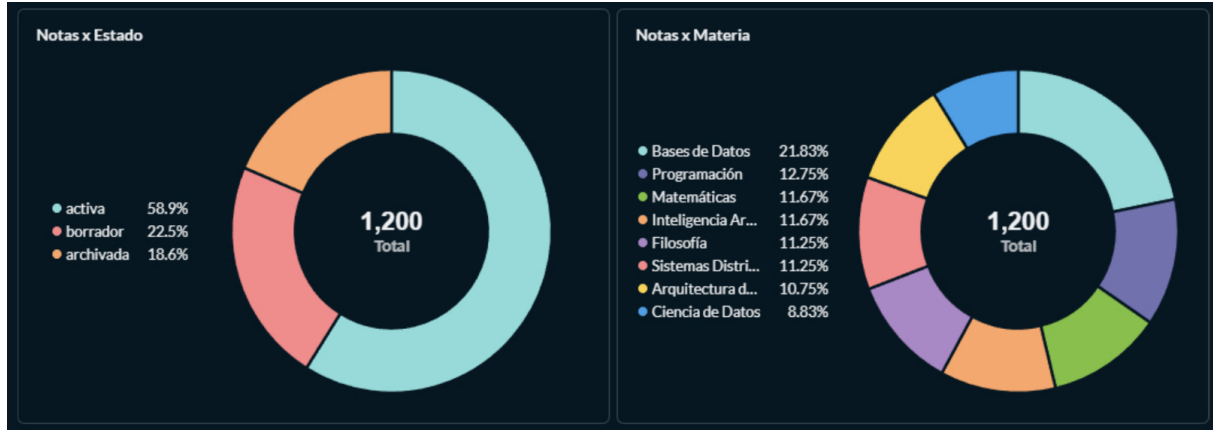


Figura 2: Distribución de notas por estado y por materia ($N = 1\,200$).

7.1. Distribución por Estado

La cohorte se reparte en tres estados canónicos: 58.9% *activa*, 22.5% *borrador* y 18.6% *archivada* (véase la figura 2). La predominancia del estado activo refleja un Zettelkasten saludable: la mayoría del corpus está en uso y vinculado, mientras que el porcentaje de *borradores* sugiere una pipeline editorial viva.

7.2. Distribución por Materia

Cuadro 1: Distribución porcentual de notas por materia ($N = 1\,200$).

Materia	Porcentaje
Bases de Datos	21.83%
Programación	12.75%
Matemáticas	11.67%
Inteligencia Artificial	11.67%
Filosofía	11.25%
Sistemas Distribuidos	11.25%
Arquitectura de Software	10.75%
Ciencia de Datos	8.83%

Bases de Datos concentra cerca de un cuarto del corpus, congruente con el dominio académico del proyecto. Las demás materias se distribuyen de forma relativamente homogénea en torno a 11%.

7.3. Top 10 de Temas

Cuadro 2: Diez temas con mayor número de notas asociadas.

Tema	Notas
Filosofía de la Ciencia	~ 39
Algoritmos	~ 36
Microservicios	~ 35
Epistemología	~ 35
Particionamiento	~ 34
Diseño de Sistemas	~ 33
ClickHouse	~ 33
Estructuras de Datos	~ 33
SQL	~ 32
Optimización	~ 31

7.4. Comparativa Interanual

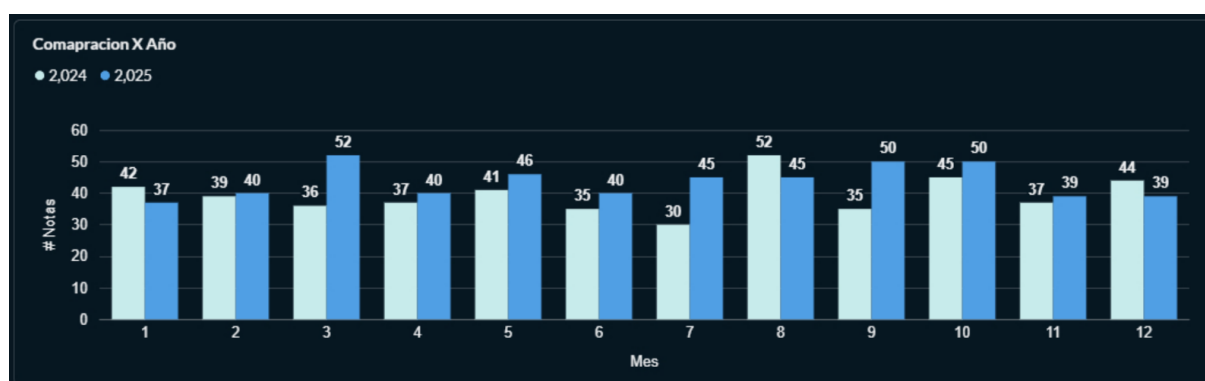


Figura 3: Comparativa mensual de notas creadas: 2024 vs. 2025.

La distribución mensual de notas entre 2024 y 2025 evidencia un crecimiento sostenido del corpus en la mayoría de los meses (figura 3). El año 2025 supera consistentemente a 2024 en nueve de doce meses, con picos en marzo (52 vs. 36) y julio (45 vs. 30), aunque agosto registra el comportamiento inverso (45 vs. 52).

7.5. Distribución de Tipos de Enlace



Figura 4: Top 10 de temas y distribución de enlaces por tipo semántico.

Cuadro 3: Distribución de *links* por tipo semántico ($N = 3000$).

Tipo de enlace	Total
prerrequisito_de	453
contradice	450
ejemplo_de	439
referencia	436
relacionado	430
inspirado_en	409
expande	383

La distribución relativamente uniforme valida el diseño del corpus sintético (sin sesgo hacia ningún tipo de enlace) y, más importante, demuestra que el sistema captura una topología semánticamente rica: la sola presencia de tipos como **contradice** o **prerrequisito_de** habilita consultas de razonamiento que no son representables en sistemas Zettelkasten convencionales basados solo en *backlinks* no tipificados (figura 4).

8. Benchmarks: Comparativa Multi-DBMS

8.1. Metodología

El script `06_benchmark_multi_dbms_v2.py` ejecuta, sobre los cuatro motores, las mismas cuatro consultas lógicas. Cada una se ejecuta en múltiples iteraciones, registrando los percentiles `avg_ms`, `min_ms` y `max_ms`. Las consultas se diseñaron de forma deliberada para favorecer, en cada caso, a un motor distinto, con el objetivo de aislar empíricamente las fortalezas específicas de cada paradigma.

Cuadro 4: Batería de consultas evaluadas.

ID	Descripción	DBMS esperado
Q1	Agregación columnar OLAP por estado, materia y año.	ClickHouse
Q2	Lectura de documento rico con datos embebidos (nota completa).	MongoDB
Q3	Recorrido de grafo de longitud variable, 1–4 saltos.	Neo4j
Q4	<i>Ranking</i> analítico sobre métricas pre-agregadas.	ClickHouse

8.2. Q1: Agregación OLAP

Cuadro 5: Q1 — Agregación OLAP por estado, materia y año.

DBMS	avg (ms)	min (ms)	max (ms)
ClickHouse	17.282	17.282	17.282
PostgreSQL	23.286	23.286	23.286
Neo4j	25.133	25.133	25.133
MongoDB	32.657	32.657	32.657

La consulta Q1 evalúa operaciones OLAP clásicas de agregación agrupando notas por estado, materia y año de creación. El mejor desempeño corresponde a ClickHouse, cuya arquitectura columnar y ejecución vectorizada están optimizadas para cargas analíticas. PostgreSQL y Neo4j muestran tiempos competitivos, mientras que MongoDB presenta un mayor costo debido al procesamiento documental y la necesidad de agregaciones sobre estructuras JSON.

8.3. Q2: Documento Rico

Cuadro 6: Q2 — Recuperación de documento enriquecido con relaciones embebidas.

DBMS	avg (ms)	min (ms)	max (ms)
ClickHouse	17.117	17.117	17.117
MongoDB	48.390	48.390	48.390
Neo4j	73.890	73.890	73.890
PostgreSQL	702.585	702.585	702.585

La consulta Q2 recupera una nota completa junto con autor, materia, tema, *tags*, fuentes, enlaces, *backlinks*, versiones y eventos asociados. MongoDB obtiene un desempeño significativamente mejor que PostgreSQL gracias al principio *store together what you read together*: toda la información relevante se encuentra embebida en un único documento JSON. PostgreSQL requiere ejecutar múltiples JOIN para reconstruir la misma estructura. Aunque ClickHouse obtiene el menor tiempo absoluto, el resultado proviene de una tabla analítica pre-agregada y no de una reconstrucción documental completa, por lo que MongoDB representa la solución conceptual más adecuada para este tipo de consulta.

8.4. Q3: Recorrido de Grafo

Cuadro 7: Q3 — Recorrido variable [: *LINK* * 1..4] desde una nota arbitraria.

DBMS	avg (ms)	min (ms)	max (ms)
Neo4j	16.281	16.281	16.281
PostgreSQL	17.911	17.911	17.911
MongoDB	31.690	31.690	31.690
ClickHouse	—	—	—

La consulta Q3 mide recorridos sobre relaciones entre notas utilizando una profundidad variable de uno a cuatro saltos. Neo4j obtiene el mejor desempeño debido a que su modelo de grafos implementa recorridos como operación nativa. PostgreSQL puede expresar la consulta mediante **WITH RECURSIVE**, aunque el costo aumenta conforme crece la profundidad del recorrido. MongoDB requiere operadores como **\$graphLookup**, con mayor complejidad y menor eficiencia. ClickHouse no fue considerado para esta consulta debido a que los recorridos recursivos no forman parte del modelo natural de una base de datos columnar orientada a analítica.

8.5. Q4: Ranking Analítico

Cuadro 8: Q4 — *Ranking* sobre métricas pre-agregadas.

DBMS	avg (ms)	min (ms)	max (ms)
ClickHouse	21.119	21.119	21.119
MongoDB	31.404	31.404	31.404
Neo4j	34.492	34.492	34.492
PostgreSQL	193.267	193.267	193.267

La consulta Q4 evalúa operaciones de *ranking* y agregación sobre métricas previamente materializadas durante el proceso ETL. ClickHouse obtiene el mejor desempeño debido a que trabaja directamente sobre tablas columnarizadas y pre-agregadas, evitando JOIN costosos y minimizando el volumen de datos leídos. MongoDB y Neo4j mantienen tiempos relativamente bajos gracias a sus estructuras flexibles, mientras que PostgreSQL presenta un mayor costo asociado a operaciones relacionales y agregaciones tradicionales.

8.6. Síntesis del Benchmark

Cuadro 9: Síntesis del benchmark: motor con mejor desempeño por tipo de consulta.

Consulta	Justificación	Ganador
Q1	Lectura columnar y agregación vectorizada para cargas OLAP	ClickHouse
Q2	Recuperación eficiente de documentos JSON enriquecidos y embebidos	MongoDB
Q3	Recorridos nativos sobre relaciones tipificadas de grafo	Neo4j
Q4	Ejecución analítica sobre tablas pre-agregadas y materializadas	ClickHouse

Los resultados demuestran que no existe un DBMS universalmente superior, sino motores especializados para distintos patrones de consulta. PostgreSQL funciona adecuadamente como sistema relacional principal y fuente de verdad transaccional; MongoDB destaca en la recuperación de documentos complejos; Neo4j sobresale en recorridos de grafos y relaciones; y ClickHouse domina las cargas analíticas y agregaciones OLAP gracias a su arquitectura columnar.

9. Conclusiones

El presente proyecto valida, sobre un caso de uso concreto, la tesis central de la *Polyglot Persistence*: no existe un motor de bases de datos universalmente óptimo. Cada paradigma codifica supuestos de diseño (estructura de almacenamiento, modelo lógico, motor de ejecución) que lo hacen sobresaliente en un perfil de carga y mediocre en otro. Las conclusiones específicas son las siguientes:

1. **PostgreSQL como *system of record*.** La capa relacional aporta garantías ACID, integridad referencial y un esquema rígido que sirve de fuente única de verdad. Sin embargo, su rendimiento se degrada sobre cargas analíticas masivas y sobre recorridos recursivos profundos.
2. **MongoDB para acceso documento-céntrico.** Para el caso de uso *document fetch* (renderizar una nota con todos sus metadatos), MongoDB elimina el costo de los JOIN mediante la denormalización controlada al momento de la ingesta. El precio a pagar es una redundancia parcial y la necesidad de un proceso ETL idempotente.
3. **Neo4j para razonamiento sobre topología.** Cualquier consulta que se exprese naturalmente como un recorrido de grafo (centralidad, prerequisites, comunidades, caminos más cortos) encuentra en Neo4j un costo proporcional al subgrafo explorado, no al tamaño total del corpus. Su Cypher expresa con economía lo que PostgreSQL articulara mediante CTE recursivas costosas.
4. **ClickHouse para analítica de alto volumen.** El modelo columnar con MergeTree es claramente superior para consultas agregadas sobre dimensiones de baja cardinalidad. La preagregación al momento del ETL traslada el costo computacional desde el

momento de consulta hacia el momento de carga, lo que es deseable cuando la frecuencia de lectura excede ampliamente a la de escritura.

5. **Coste arquitectónico.** La políglota no es gratuita. Cada motor adicional implica un proceso ETL, un esquema de monitoreo, una política de respaldo y un costo cognitivo para el equipo. La inversión se justifica solo cuando los patrones de consulta son suficientemente heterogéneos: precisamente el caso de un sistema Zettelkasten.
6. **Reproducibilidad.** La combinación de Docker Compose, semilla aleatoria fija y scripts ETL versionados garantiza que el experimento sea reproducible en cualquier entorno con un único comando `docker compose up`. Esto convierte al sistema en un *benchmark* replicable, no en un caso aislado.

En conjunto, el proyecto demuestra que las decisiones de almacenamiento no deben tomarse de forma global, sino caso por caso, y que el costo de mantener múltiples motores se amortiza con creces cuando cada uno se emplea estrictamente para el tipo de consulta que mejor sabe responder.

Referencias

- [1] Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional.
- [2] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [3] The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
- [4] MongoDB, Inc. (2024). *MongoDB 7.0 Manual*. <https://www.mongodb.com/docs/manual/>
- [5] Neo4j, Inc. (2024). *Neo4j 5 Documentation*. <https://neo4j.com/docs/>
- [6] ClickHouse, Inc. (2024). *ClickHouse Documentation*. <https://clickhouse.com/docs>
- [7] Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph Databases: New Opportunities for Connected Data* (2nd ed.). O'Reilly Media.
- [8] Metabase, Inc. (2024). *Metabase Documentation*. <https://www.metabase.com/docs/>